

0.1 Question 1: Managing data files

We will use data that is available from NBA. Although NBA doesn't officially make the data API (application programming interface) public, people have figured out ways to access their data programmatically (1, 2). However, NBA does not offer an official API and it is possible to get our JupyterHub blocked by the site if we use them. Therefore, in this assignment, the raw data downloads are provided to you in a zip file: <https://ucsb.box.com/shared/static/z6y3etgikbzbfn0ld4brvc95xtgjcrie.zip>

Download and unzip the file to a directory named **data** using command line commands (unzipping on Windows and Mac may not work because different OS have different constraints on filename lengths, etc.). Adding an exclamation point in the Jupyter notebook cell indicates that **bash** shell interpreter will execute your command.

```
wget -nc https://ucsb.box.com/shared/static/z6y3etgikbzbfn0ld4brvc95xtgjcrie.zip -O nba-data.zip
unzip -o nba-data.zip -d data
```

What these commands are doing: * **wget** downloads files ([what do each of the pieces do?](#)) * **unzip** will unzip **nba-data.zip** into a directory named **data** (specified by **-d data**) and will overwrite any same filenames when extracting (specified by **-o**).

```
In [2]: !wget -nc https://ucsb.box.com/shared/static/z6y3etgikbzbfn0ld4brvc95xtgjcrie.zip -O nba-data.zip
        !unzip -o nba-data.zip -d data
```

File 'nba-data.zip' already there; not retrieving.

Archive: nba-data.zip

```
  inflating: data/commonallplayers?LeagueID=00&Season=2018-19&IsOnlyCurrentSeason=0
  inflating: data/commonTeamYears?LeagueID=00&Season=2018-19
  inflating: data/shotchartdetail?PlayerID=101106&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101107&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101108&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101109&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101112&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101123&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101133&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101139&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101141&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101150&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101161&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=101181&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626143&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626144&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626145&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626147&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626149&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626153&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626154&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
  inflating: data/shotchartdetail?PlayerID=1626155&PlayerPosition=&Season=2018-19&ContextMeasure=FGA&Date=2018-11-01
```

[illegible]

[illegible]

[illegible]

0.1.1 Question 2b: All Shots Scatter Plot + Court Outline

Again use seaborn to make a scatter plot of Stephen Curry's shots. Again, set the x-axis limits to span (-300, 300), the y-axis limits to span (-100, 500) color the points by whether the shot was made or missed. Set the missed shots to have an 'x' symbol and made shots to be a circular symbol. Call the `draw_court` function with `outer_lines` set to to be true. Save the `Axes` returned by the plot call in a variable called `ax`.

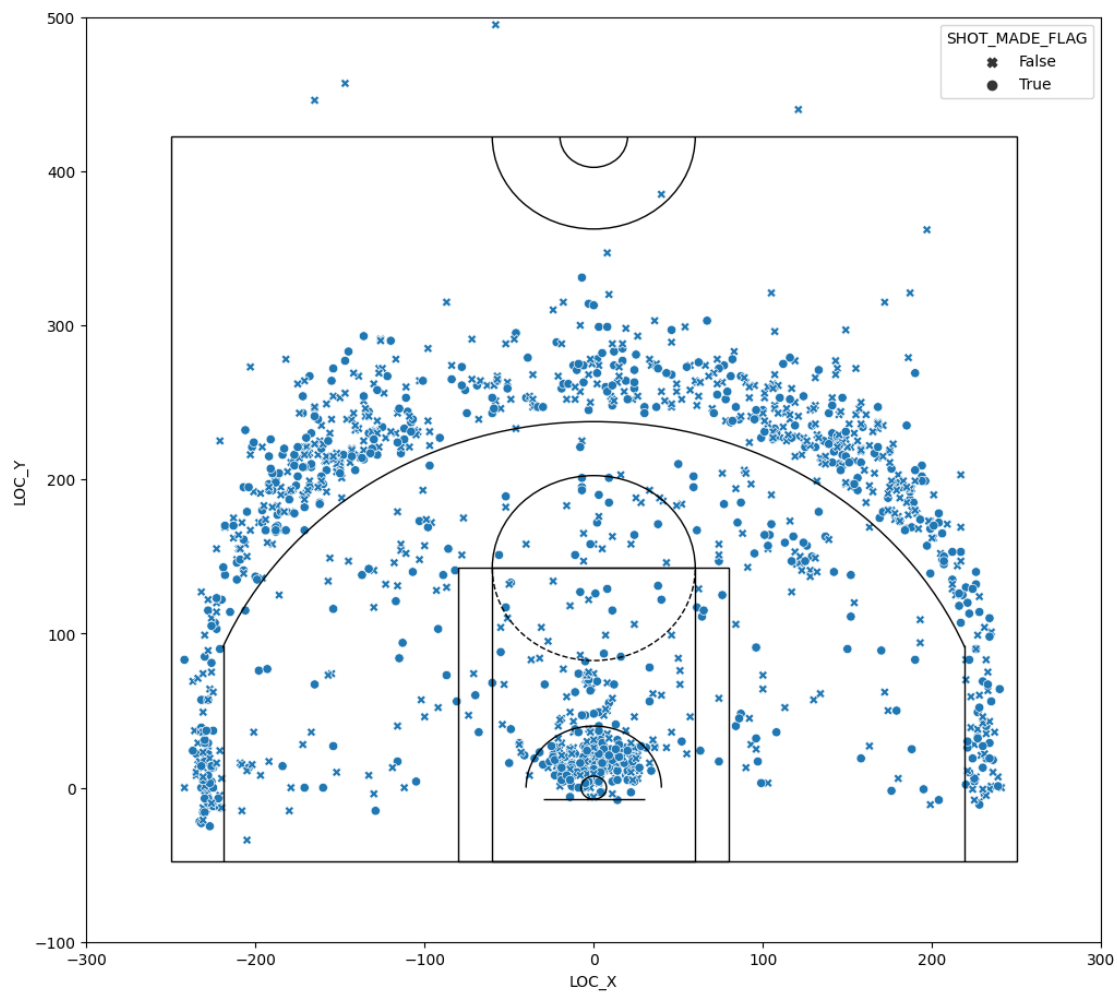
```
In [18]: plt.figure(figsize=(12, 11))
         markers = {0 : "X", 1 : "o"}
         ax = sns.scatterplot(x='LOC_X', y='LOC_Y', style='SHOT_MADE_FLAG', markers=markers, data=curry)

         ax.set_xlim(-300, 300)
         ax.set_ylim(-100, 500)

         ax.set_xlabel('LOC_X')
         ax.set_ylabel('LOC_Y')
         draw_court(outer_lines=True, ax=ax)

         plt.show()
```

```
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use pandas.api.types.is_categorical_dtype instead
  if pd.api.types.is_categorical_dtype(vector):
```



0.1.2 Question 2c: Analyzing the Visualization

In a few sentences, discuss what makes this an effective or ineffective visualization for understanding the types of shots that Stephen Curry likes to take and is good at taking, relative to other players in the league. Are there ways it can be improved?

SOLUTION

The scatterplot visualization effectively maps Stephen Curry's shot locations and outcomes on a basketball court. It provides clear spatial analysis of his shooting patterns. However, its effectiveness is limited without comparison to other players or league averages. Potential enhancements are color coding, adding a density overlay to indicate high-frequency shooting areas, and incorporating shooting percentage data.

0.1.3 Question 2d: A Hexbin plot

Visualize Stephen Curry's shots by using a [hexbin plot with marginal histograms](#). Also refer to setting [figure aesthetics](#) for what commands below do.

```
In [19]: sns.set_style("white")
         joint_shot_chart = sns.jointplot(x="LOC_X", y="LOC_Y", data=curry_data,
                                         kind='hex', color='blue')

         joint_shot_chart.fig.set_size_inches(12,11)

         # A joint plot has 3 Axes, the first one called ax_joint
         # is the one we want to draw our court onto and adjust some other settings
         ax = joint_shot_chart.ax_joint
         draw_court(ax, outer_lines=True)

         # Adjust the axis limits and orientation of the plot in order
         # to plot half court, with the hoop by the top of the plot
         ax.set_xlim(-300, 300)
         ax.set_ylim(500, -100)

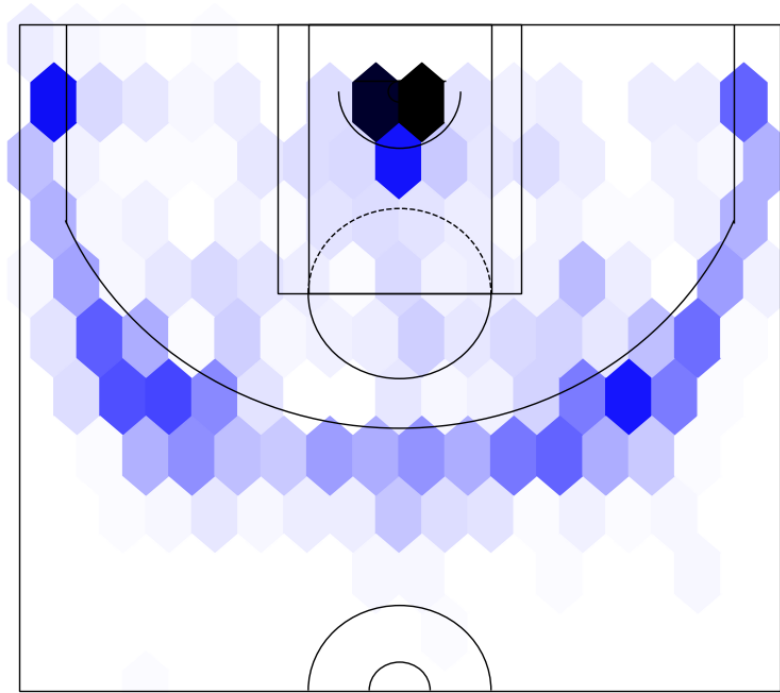
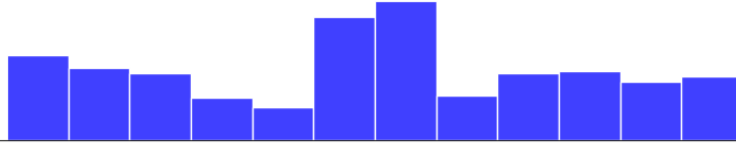
         # Get rid of axis labels and tick marks
         ax.set_xlabel('')
         ax.set_ylabel('')
         ax.tick_params(labelbottom=False, labelleft=False)

         # Add a title
         ax.set_title('Stephen Curry, 2018-19, FGA',
                      y=1.2, fontsize=10)

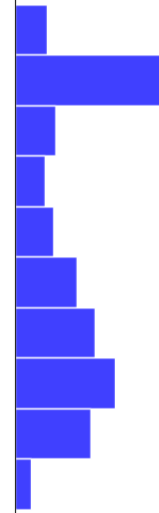
         # Add Data Source and Author
         ax.text(-250,445,'\n This plot is based on code by Savvas Tjortjoglou (savvastjortjoglou.com)'
                 fontsize=12);

/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Use na_values=[None] instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead.
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Use na_values=[None] instead.
  with pd.option_context('mode.use_inf_as_na', True):
```


Stephen Curry, 2018-19, FGA



This plot is based on code by Savvas Tjortjoglou (savvastjortjoglou.com)



0.1.4 Question 3b: Visualize the binning on curry_data

Call `bin_shots` on `curry_data` to create a binned but unsmoothed matrix of shot counts (call this `curry_binned_unsmoothed`), a binned and smoothed matrix of counts with `sigma=1` (call this `curry_binned_smoothed1`) and one with `sigma=5` (call this `curry_binned_smoothed5`). Use the bin edges defined below:

```
In [22]: ## bin edge definitions in inches
xedges = np.linspace(start=-300, stop=300, num=151)
yedges = np.linspace(start=-48, stop=372, num=106)
```

```
In [23]: bin_edges = (xedges, yedges)
```

```
curry_binned_unsmoothed, xe, ye = bin_shots(curry_data, bin_edges, density=False, sigma=0)
curry_binned_smoothed1, xe, ye = bin_shots(curry_data, bin_edges, density=True, sigma=1)
curry_binned_smoothed5, xe, ye = bin_shots(curry_data, bin_edges, density=True, sigma=5)
```

Visualize the shots as a heatmap

```
In [24]: def plot_shotchart(binned_counts, xedges, yedges, ax=None, use_log=False, cmap = 'Reds'):

    """Plots 2d heatmap from vectorized heatmap counts

    Args:
        hist_counts: vectorized output of numpy.histogram2d
        xedges, yedges: bin edges in arrays
        ax: figure axes [None]
        use_log: will convert count x to log(x+1) to increase visibility [False]
        cmap: Set the color map https://matplotlib.org/examples/color/colormaps\_reference.html

    Returns:
        ax: axes with plot
    """

    import numpy as np
    import matplotlib.pyplot as plt

    ## number of x and y bins.
    nx = xedges.size - 1
    ny = yedges.size - 1

    X, Y = np.meshgrid(xedges, yedges)

    if use_log:
        counts = np.log(binned_counts + 1)

    if ax is None:
        fig, ax = plt.subplots(1,1)
```

```

ax.pcolormesh(X, Y, binned_counts.T, cmap=cmap)
ax.set_aspect('equal')

draw_court(ax)

return(ax)

```

Plot three plots of `curry_binned_unsmoothed`, `curry_binned_smoothed1` and `curry_binned_smoothed5`.

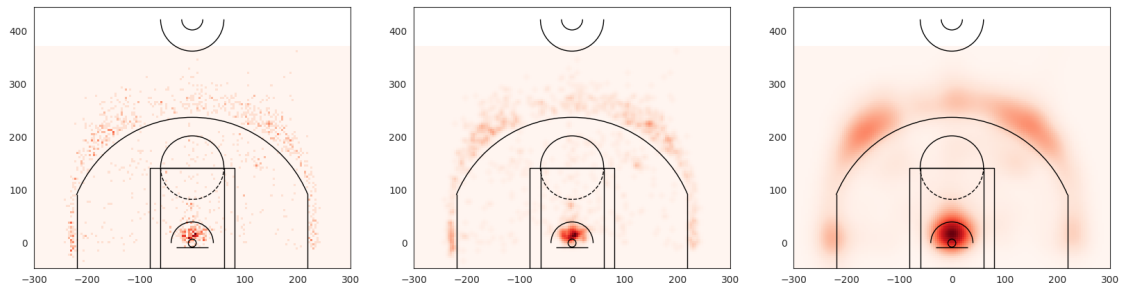
```

In [25]: fig, ax = plt.subplots(1, 3, figsize=(20,60))

plot_shotchart(curry_binned_unsmoothed, xe, ye, ax=ax[0], use_log=False, cmap='Reds')
plot_shotchart(curry_binned_smoothed1, xe, ye, ax=ax[1], use_log=False, cmap='Reds')
plot_shotchart(curry_binned_smoothed5, xe, ye, ax=ax[2], use_log=False, cmap='Reds')

fig.show()

```



0.1.5 Question 4b: Visualizing Shot Types

Plot the first three basis images by calling `plot_vectorized_shot_chart` below on the columns of `W3`.

```
In [30]: def plot_vectorized_shotchart(vec_counts, xedges, yedges, ax=None, use_log=False, cmap = 'Reds')
        """Plots 2d heatmap from vectorized heatmap counts

        Args:
            hist_counts: vectorized output of numpy.histogram2d
            xedges, yedges: bin edges in arrays
            ax: figure axes [None]
            use_log: will convert count x to log(x+1) to increase visibility [False]
            cmap: Set the color map https://matplotlib.org/examples/color/colormaps\_reference.html
        Returns:
            ax: axes with plot
        """

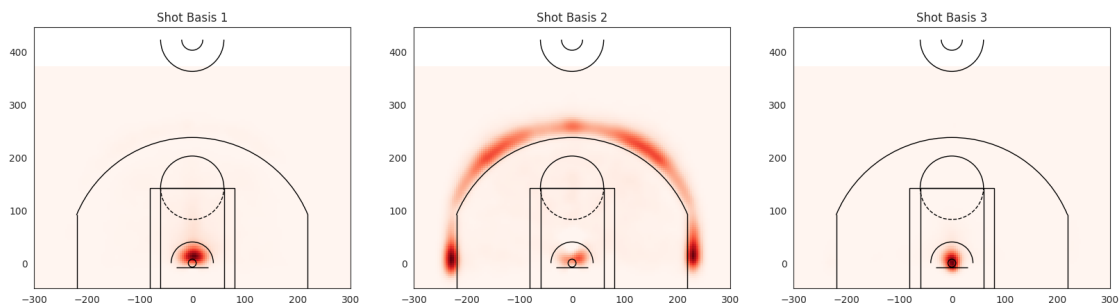
        nx = len(xedges) - 1
        ny = len(yedges) - 1

        # use reshape to convert a vectorized counts back into a 2d histogram
        two_d_counts = vec_counts.reshape(nx, ny)

        return(plot_shotchart(two_d_counts, xedges, yedges, ax=ax, use_log=use_log, cmap=cmap))

fig, ax = plt.subplots(1, 3, figsize=(20,60))

## Write a for loop
for i in range(3):
    plot_vectorized_shotchart(W3[:, i], xedges, yedges, ax=ax[i], use_log=False, cmap='Reds')
    ax[i].set_title('Shot Basis %i' % (i+1))
```



0.1.6 Question 4c: Reconstruction Error

Below we re-construct the shooting pattern for a single player. By “reconstructing” we mean use the approximation

$$\hat{X} = WH$$

obtained via NMF. Find $\hat{X}$ by multiplying W and H. In python the @ symbol is used for matrix multiplication.

Type your answer here, replacing this text.

```
In [31]: X3_hat = W3 @ H3
```

```
In [32]: np.where(pids == 1629013)
```

```
/tmp/ipykernel_6818/3357615216.py:1: DeprecationWarning: Calling nonzero on 0d arrays is deprecated, as
np.where(pids == 1629013)
```

```
Out[32]: (array([], dtype=int64),)
```

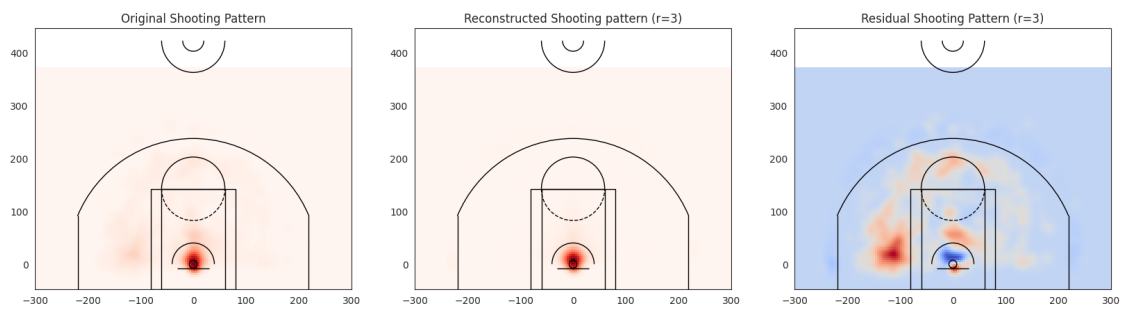
```
In [33]: # Find the player_id of LaMarcus Aldridge
player_id = allshots[allshots.PLAYER_NAME == 'LaMarcus Aldridge'].index[0]

## find index in X corresponding to that player
to_plot_idx = np.where(np.array(pids) == player_id)[0][0]

fig, ax = plt.subplots(1, 3, figsize=(20,60))

## Call plot_vectorized_shot_chart
original_shotchart = plot_vectorized_shotchart(X[:,to_plot_idx],xedges,yedges,ax = ax[0])
reconstructed_shotcfhart = plot_vectorized_shotchart(X3_hat[:,to_plot_idx],xedges,yedges, ax =
residual_chart = plot_vectorized_shotchart((X-X3_hat)[:, to_plot_idx], xedges,yedges,ax = ax[2])

# print(max(abs(X3_hat[:, to_plot_idx] - X[:, to_plot_idx])))
ax[0].set_title('Original Shooting Pattern')
ax[1].set_title('Reconstructed Shooting pattern (r=3)')
ax[2].set_title('Residual Shooting Pattern (r=3)')
None # prevents the title string from showing up as output
```



0.1.7 Question 4d: Choice of Colormap

Why does it make sense to use a *sequential* palette for the original and reconstructed shot charts and a *diverging* palette for the residual? *Hint:* Read the introduction to colormaps [here](#).

SOLUTION

Sequential palettes are more suitable for data with a natural sequence, whereas diverging palettes are ideal for data centered around a pivotal value, such as figures fluctuating around zero.

What areas of the court does this player shoot more and where less relative to the reconstructed area. If its helpful, you can refer to court locations by name using this legend [here](#).

SOLUTION

The player takes shots from the left side of the basket, usually within the radius of the center circle, and rarely attempts three-pointers.

0.1.8 Question 4e: More Detailed Modeling

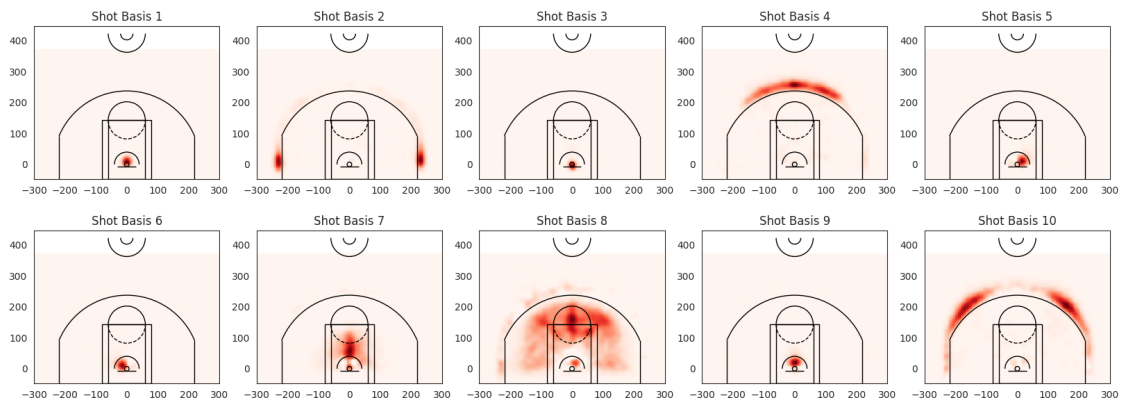
Re-run the analysis, this time for 10 basis vectors instead of 3. Again plot the bases using `plot_vectorized_shotchart` on the columns of `W10`.

Hint: Study the following code

```
fig, ax = plt.subplots(2, 5, figsize=(20, 7))
ax = ax.flatten() # turn ax into a flat array
ax[0].set_title('hello')
ax[9].set_title('there')
fig.show()
```

```
In [34]: W10, H10 = non_negative_marix_decomp(10, X)
fig, ax = plt.subplots(2, 5, figsize=(20, 7))

## Write a for loop
for i in range(10):
    plot_vectorized_shotchart(W10[:,i], xedges, yedges, ax= ax[i//5, i % 5])
    ax[i//5, i % 5].set_title('Shot Basis %i' % (i+1))
```



If you did things correctly, you should be really impressed! We've identified potentially interesting patterns of shooting styles without actually specifying anything about the way basketball is played or where the relevant lines are on the court. The resulting images are based only on the actual behavior of the players. Even more impressive is that we're capturing similarity in regions that are far apart on the court. One reason we can do this is that a basketball court is symmetric along the length of the court (i.e. symmetric about $x=0$). However, people tend to be left or right hand dominant, which might affect their preferences. Look carefully at the shot basis plots above: is there any evidence of *asymmetry* in player shooting behavior? Refer to specific basis images in your answer.

SOLUTION


```
In [35]: X10_hat = W10 @ H10
```

```
fig, ax = plt.subplots(1, 3, figsize=(20,60))
```

```
# I took the first player appearing in first column  
# (you probably want to do more interesting players)
```

```
original_shotchart = plot_vectorized_shotchart(X[:,to_plot_idx], xedges, yedges, ax=ax[0])
```

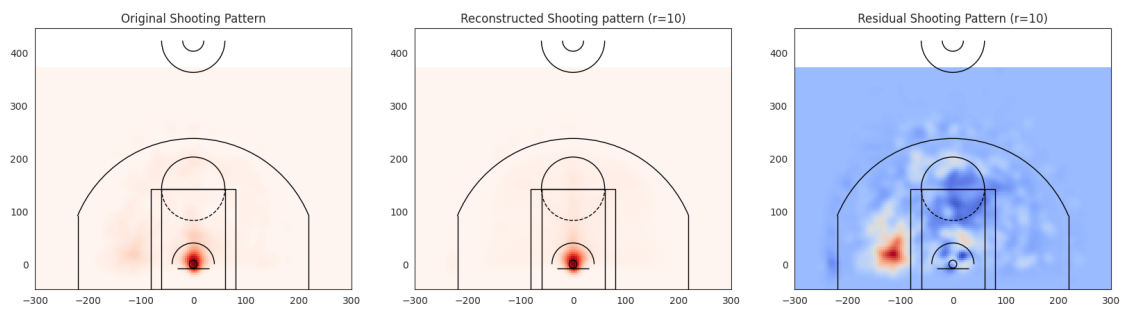
```
reconstructed_shotchart = plot_vectorized_shotchart(X10_hat[:,to_plot_idx], xedges, yedges, ax=ax[1])
```

```
residual_chart = plot_vectorized_shotchart((X-X10_hat)[:,to_plot_idx], xedges, yedges, ax=ax[2])
```

```
ax[0].set_title('Original Shooting Pattern')
```

```
ax[1].set_title('Reconstructed Shooting pattern (r=10)')
```

```
ax[2].set_title('Residual Shooting Pattern (r=10)');
```



0.1.9 Question 4f: Comparing Players

With H10 matrix, it is possible to compare any pair of players. For all players pairwise, i and j , compare using euclidean distance between their coefficients:

$$\text{player-distance}(i, j) = \|H_i - H_j\|_2 = \left(\sum_{k=1}^{10} (H_{ki} - H_{kj})^2 \right)^{1/2}$$

Create a heatmap for comparing pair-wise player distance matrix. Find the two pairs of players with the smallest distances. Also, find two pairs of players with largest distances.

Hint: you can construct the distance matrix manually or use `scipy.spatial.distance_matrix`.

SOLUTION

```
In [36]: from scipy.spatial import distance_matrix
import seaborn as sns

dist_matrix = distance_matrix(H10.T, H10.T)

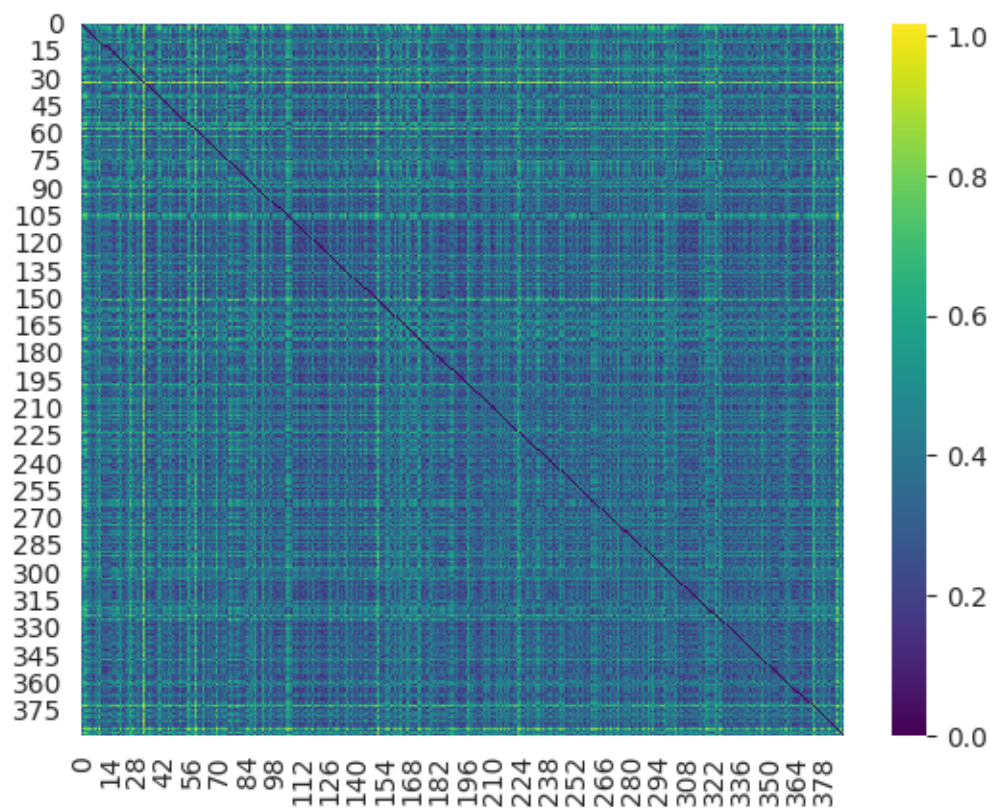
sns.heatmap(dist_matrix, cmap='viridis')
np.fill_diagonal(dist_matrix, np.inf)
min_indices = np.unravel_index(np.argmin(dist_matrix, axis=None), dist_matrix.shape)
max_indices = np.unravel_index(np.argmax(dist_matrix, axis=None), dist_matrix.shape)

players_closest = (pids[min_indices[0]], pids[min_indices[1]])
players_farthest = (pids[max_indices[0]], pids[max_indices[1]])

print("Player with the smallest distance:", players_closest)
print("Player with the largest distance:", players_farthest)
```

Player with the smallest distance: (203468, 1627750)

Player with the largest distance: (1713, 1713)



0.1.10 Question 4g: Residuals

The residual between $\hat{X}$ and X gives a sense of how well a player is described by NMF computed matrices W and H . Calculate RMSE for each player, and plot the histogram. Comment on this distribution and players with smallest and largest RMSEs (use 10 components).

SOLUTION

```
In [37]: import matplotlib.pyplot as plt

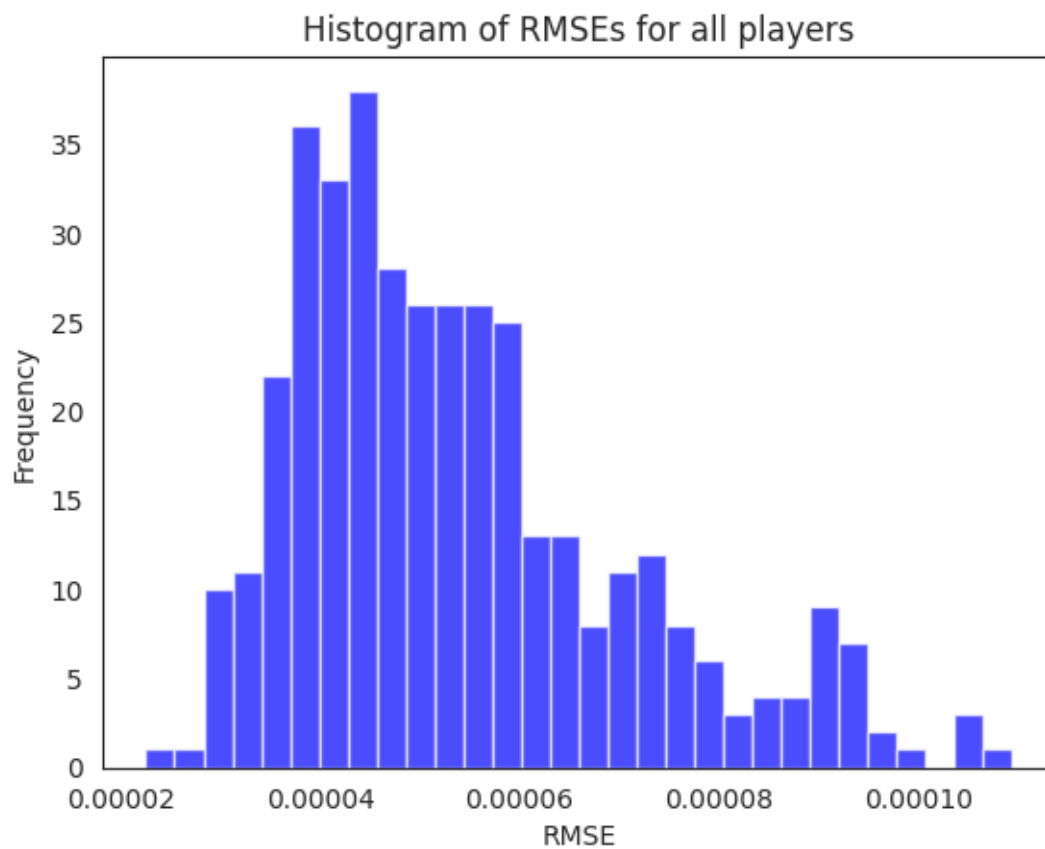
rmse_list = []
for i in range(X.shape[1]):
    rmse = np.sqrt(np.mean((X10_hat[:, i] - X[:, i]) ** 2))
    rmse_list.append(rmse)

rmse_array = np.array(rmse_list)

plt.hist(rmse_array, bins=30, alpha=0.7, color='blue')
plt.title('Histogram of RMSEs for all players')
plt.xlabel('RMSE')
plt.ylabel('Frequency')
plt.show()

min_rmse_idx = np.argmin(rmse_array)
max_rmse_idx = np.argmax(rmse_array)

print("Player with the smallest RMSE", pids[min_rmse_idx], rmse_array[min_rmse_idx])
print("Player with the largest RMSE", pids[max_rmse_idx], rmse_array[max_rmse_idx])
```



Player with the smallest RMSE 1626149 2.2442358312563833e-05
Player with the largest RMSE 202954 0.00010946061707727094

0.1.11 Question 4h: Proposing improvements

One of the main purposes of exploratory data analysis is to generate new ideas, directions, and hypothesis for future analyses and experiments. Take two players of your choice and compare their shooting patterns with various visualizations.

State any insights and defend your conclusions with visual and/or numerical comparisons.

SOLUTION

